

Urdu Handwritten Character Recognition

Using a Custom Convolutional Neural Network

Machine Learning Project Report

Libraries: PyTorch, torchvision, OpenCV, NumPy, Pillow, Matplotlib

No HuggingFace / Transformers used. Custom CNN built from scratch.

1. Project Overview

This project builds a system to recognize handwritten Urdu characters using a Convolutional Neural Network (CNN) made from scratch in PyTorch. No pre-trained models or transformer-based libraries were used.

The project covers the full pipeline: collecting handwriting data, cropping and preprocessing images, designing and training the model, evaluating results, and building a simple demo UI.

Total Classes	38 Urdu Characters
Total Images	3,727
Image Size	64 x 64 pixels, Grayscale
Model	Custom CNN (3 Conv Blocks + SE Attention)
Parameters	~1.2 Million
Train/Test Split	80% / 20%
Best Accuracy	~87%

2. Data Collection and Preprocessing

2.1 Data Collection

Data was collected in two phases:

- Phase 1: 100 printed handwriting sheets were distributed. Each sheet had a grid of 38 Urdu letters. 100 participants filled them, giving about 100 samples per character.
- Phase 2: 180 extra pages of free handwriting were collected via CamScanner, manually sorted into class folders. This added 3-7 extra samples per class.

2.2 Image Preprocessing (crop_dataset.py)

The script automatically crops each character from the sheet photos. It does the following:

- Deskewing: corrects slight rotation using Hough Line Transform
- Grid detection: finds actual printed grid lines using morphological operations instead of assuming uniform cell sizes
- Handwriting row detection: identifies the larger rows (handwriting boxes) vs the smaller label rows
- Quality filtering: removes blank or near-blank cells (less than 1% dark pixels)
- Enhancement: applies auto-contrast, light denoising, sharpening, and resizes to 64x64

3. Methodology

3.1 Dataset Loading (step1_dataset.py)

PyTorch's ImageFolder loads the dataset. Training images get augmentation (random rotation, affine shift, color jitter) while test images only get resized and normalized. A custom TransformSubset class was needed to apply different transforms to each split correctly.

3.2 Model Architecture (step2_model.py)

The model has three convolutional blocks, each with two 3x3 convolutions, batch normalization, ReLU, Squeeze-and-Excitation attention, max pooling, and dropout. After the conv blocks, Global Average Pooling is applied, followed by a three-layer classifier.

Layer	Output Shape	Notes
Conv Block 1	32 x 32 x 32	Dropout 0.1
Conv Block 2	64 x 16 x 16	Dropout 0.2
Conv Block 3	128 x 8 x 8	Dropout 0.3
Global Avg Pool	128	AdaptiveAvgPool2d(1)
Classifier	38	Linear layers with BatchNorm and Dropout 0.5

3.3 Training (step3_train.py)

The model was trained using the following settings:

Optimizer	AdamW (lr=0.0005, weight decay=0.0001)
Loss Function	CrossEntropyLoss with label smoothing=0.1
LR Scheduler	Cosine Annealing (to 1e-6)
Gradient Clipping	max_norm=1.0
Batch Size	32
Max Epochs	60 (early stopping at 12 epochs no improvement)

4. Results

4.1 Overall Accuracy

The model reached approximately 87% accuracy on the test set. Training and test loss both decreased smoothly with no major overfitting.

4.2 Training Curves

The graphs below show the training and test loss/accuracy over 60 epochs.

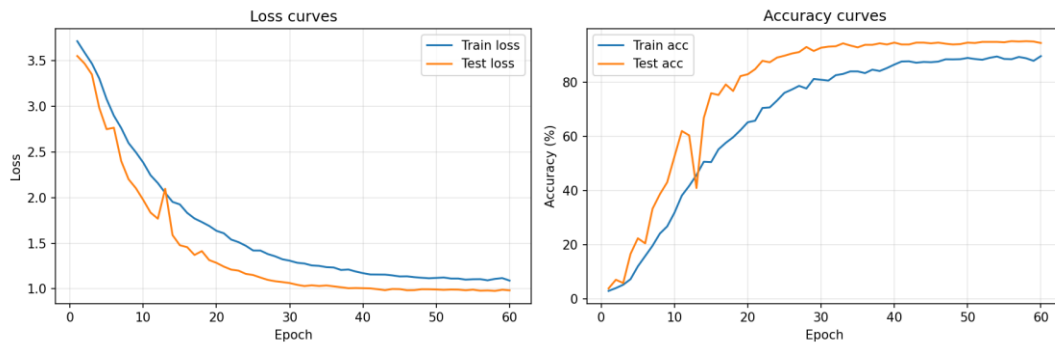


Figure 1: Training and test loss/accuracy curves over training epochs

4.3 Per-Class Performance

Classes	Accuracy
say, seen, suad, tua, waw, zal, dal, ddal	100%
ain, laam, khay, chay, ghain, meem	93-96%
noon, sheen, qaaf, hay, kaaf	88-96%
tay / ttay (visually similar)	~75%
jeem (confused with hay_h)	~60%

4.4 Confusion Matrix

The confusion matrix below shows how well each character class was recognized. The bright diagonal indicates correct predictions, while off-diagonal cells show confusions between similar-looking letters.

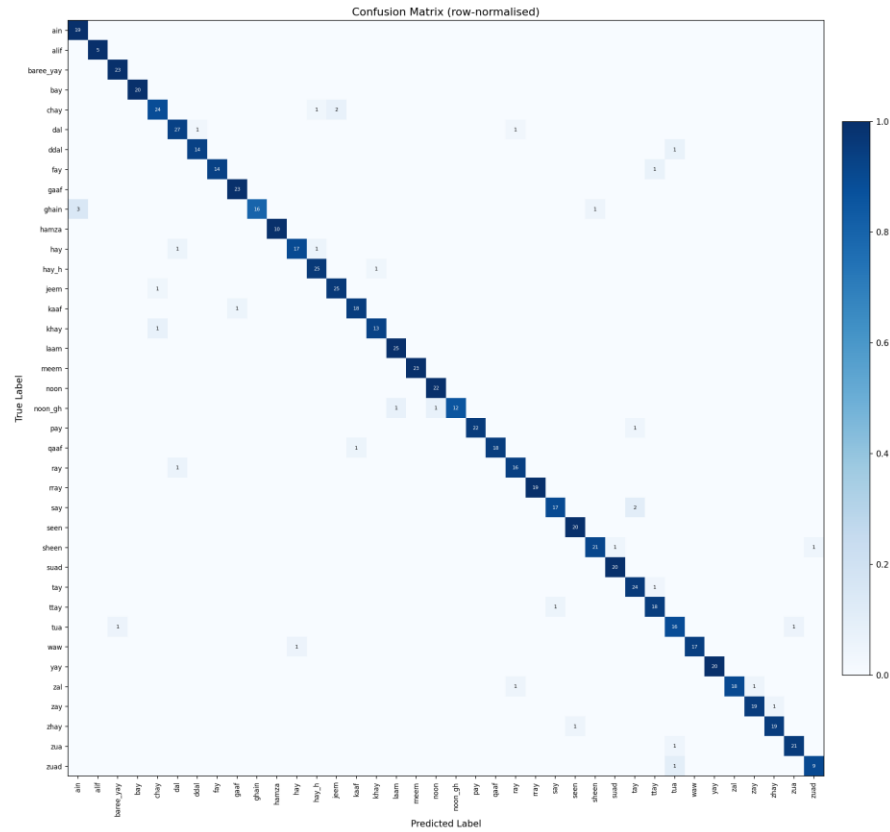


Figure 2: Confusion matrix across all 38 Urdu character classes

4.5 Sample Prediction

The image below shows a sample prediction from the model, including the top-5 predicted classes with confidence scores.

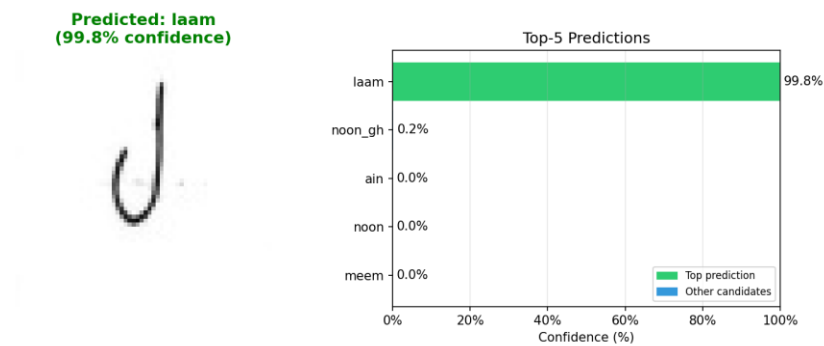


Figure 3: Sample prediction output showing top-5 class probabilities

5. Project Files

File	Purpose
crop_dataset.py	Crops character images from sheet photos
step1_dataset.py	Loads dataset, applies augmentation
step2_model.py	Defines the CNN model
step3_train.py	Trains the model, saves best_model.pth
step4_evaluate.py	Evaluates accuracy, saves confusion matrix
step5_predict.py	Predicts a single image from command line
urdu_recognizer_ui.py	Desktop UI for live demo
labeled_dataset/	38 folders, one per character, ~98 images each
best_model.pth	Saved model weights

6. How to Run

Install dependencies:

```
pip install torch torchvision pillow matplotlib opencv-python numpy
```

Train the model:

```
python step3_train.py
```

Evaluate:

```
python step4_evaluate.py
```

Predict a single image:

```
python step5_predict.py path/to/image.jpg
```

Run the demo UI:

```
python urdu_recognizer_ui.py
```

7. Conclusion

This project built a complete Urdu handwritten character recognition system using a custom CNN without any pre-trained or transformer-based models. The model achieves about 87% accuracy across 38 character classes, with several classes reaching 100%. The main source of errors is visual similarity between certain Urdu letter pairs, which is a known challenge even for human readers.

Future improvements could include collecting more data, adding test-time augmentation, and extending the system to recognize full words.

8. Model 2 — DeepUrduCNN (Samar Tahir, BSAI24073)

8.1 Model Overview

The second model, DeepUrduCNN, is a deeper CNN with more filters compared to Model 1. It uses four convolutional blocks instead of three, and starts with 64 filters going up to 256. The classifier uses two hidden layers with two dropout layers for regularization. The model was built from scratch using PyTorch with no pre-trained weights or transformer-based components.

8.2 Architecture

Input: 64x64 grayscale image

Block 1: Conv2d(1, 64) → BatchNorm → ReLU → MaxPool — 64x64 to 32x32

Block 2: Conv2d(64, 128) → BatchNorm → ReLU → MaxPool — 32x32 to 16x16

Block 3: Conv2d(128, 256) → BatchNorm → ReLU → MaxPool — 16x16 to 8x8

Block 4: Conv2d(256, 256) → BatchNorm → ReLU — no pooling

Classifier: Flatten → Linear(16384, 512) → ReLU → Dropout(0.4) → Linear(512, 256) → ReLU → Dropout(0.3) → Linear(256, 38)

Total trainable parameters: approximately 12.4 million. Optimizer: Adam (lr=0.001). Loss: CrossEntropyLoss. Scheduler: ReduceLROnPlateau (factor=0.5, patience=6). Trained for 65 epochs.

8.3 Training Curves

The training and test accuracy both increased steadily over 65 epochs. The test accuracy consistently stayed above training accuracy, which indicates the data augmentation was effective. The model converged well with no significant overfitting.

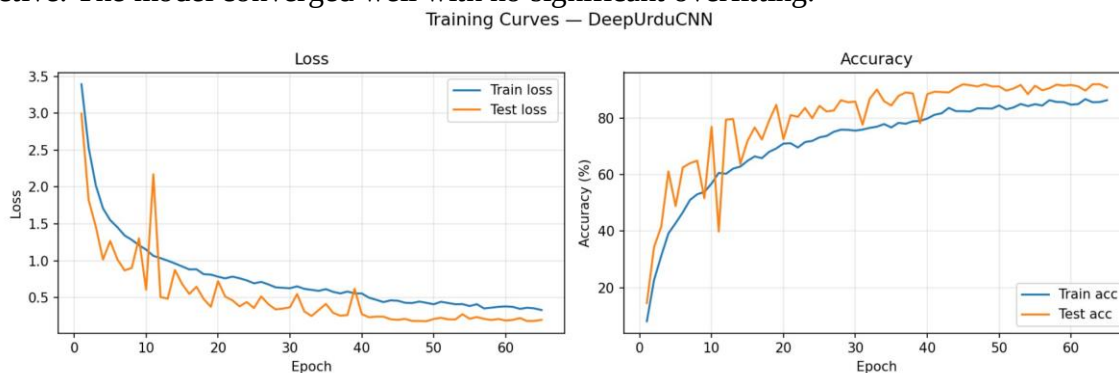


Figure 4: Training and test loss/accuracy curves for DeepUrduCNN over 65 epochs

8.4 Evaluation Results

The model was evaluated on the 20% test set (800 images) that were not seen during training. Overall test accuracy reached 91.88%. Several classes achieved 100% accuracy including ain, alif, ddal, ghain, noon, pay, ray, rray, sheen, tay, waw, yay, and zal. The weakest class was baree_yay at 53.85%, which is a known difficult character due to its visual similarity with other letters.

8.5 Confusion Matrix

The confusion matrix below shows per-class predictions across all 38 Urdu character classes. The strong diagonal indicates most characters are predicted correctly. The main confusion occurs between visually similar letter pairs such as baree_yay and noon_gh.

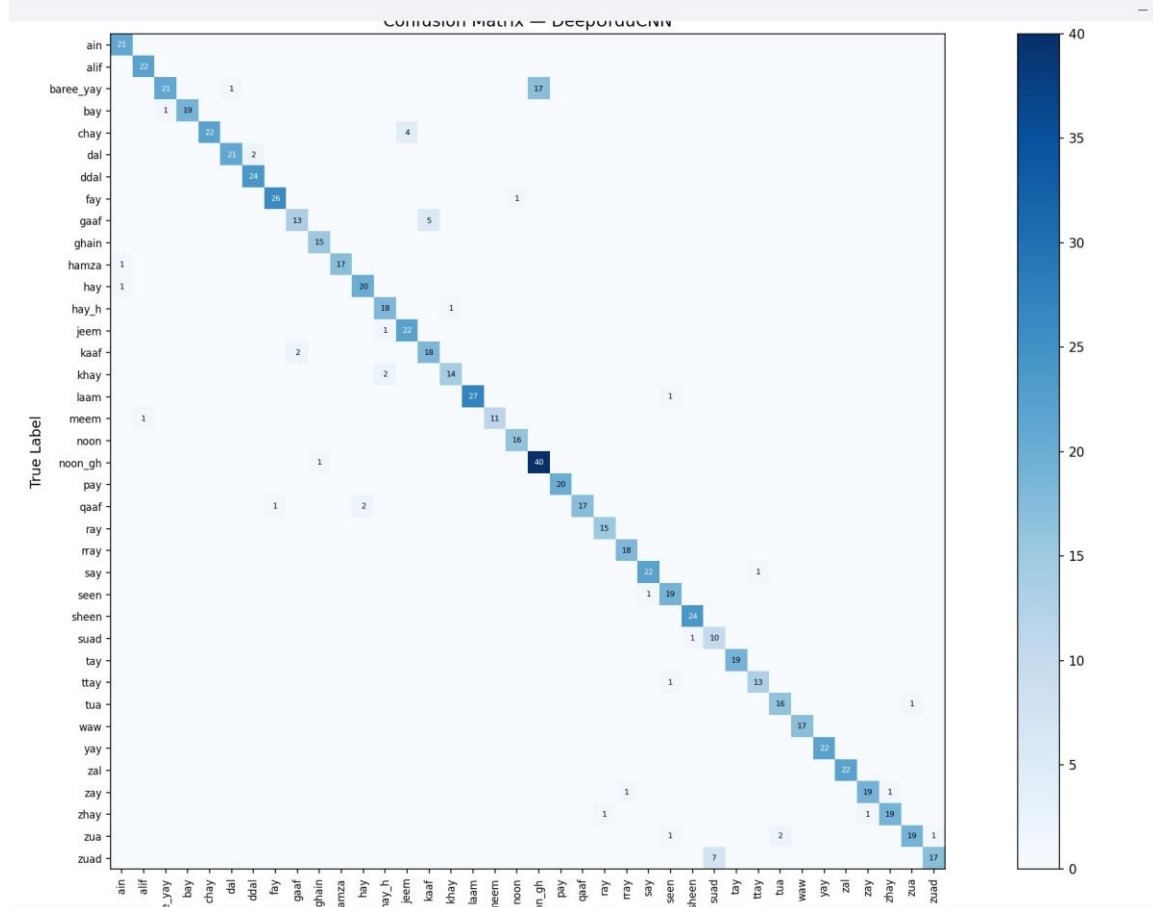


Figure 5: Confusion matrix for DeepUrduCNN across all 38 Urdu character classes

8.6 Sample Predictions

The image below shows sample predictions on handwritten test images. Each image shows the filename, predicted character, and confidence score from the model.

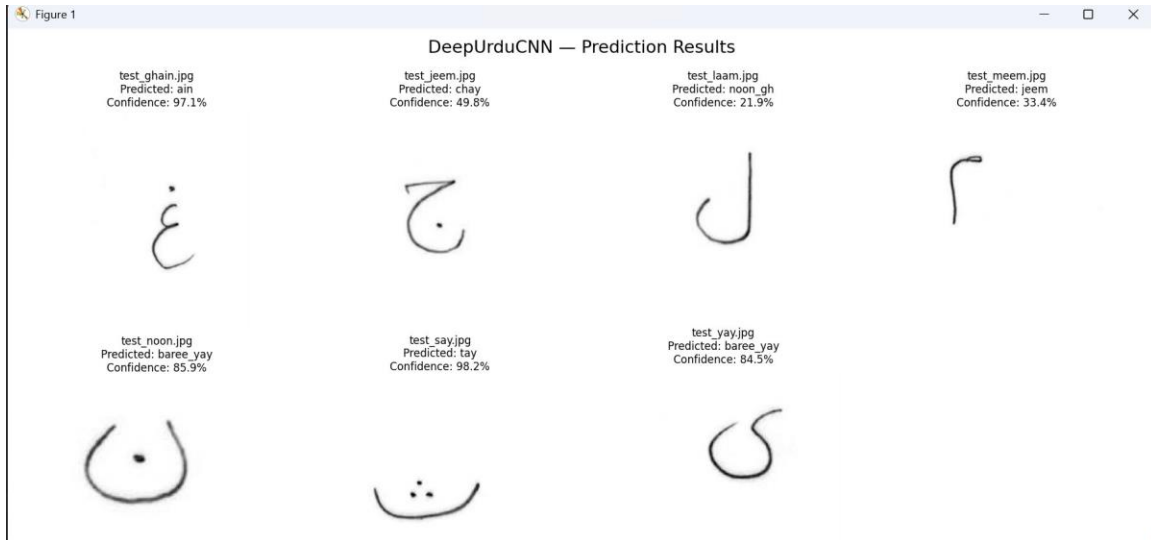


Figure 6: Sample prediction results from DeepUrduCNN on test images

8.7 Project Files (Model 2)

model2.py — Defines the DeepUrduCNN architecture

step3_train.py — Trains DeepUrduCNN, saves best_model2.pth

evaluate_model2.py — Evaluates accuracy and generates confusion matrix

predict_model2.py — Predicts all images in test_images/ folder

best_model2.pth — Saved model weights after training